ICT&L REVENUE

Revenue
Cáin agus Custaim na hÉireann
Irish Tax and Customs

VAT3 Webservice

REST Web Service Integration Guide

Table of Contents

Audience 3
Document context 3
Related Documents 3
1. Introduction 4
2. Calling the Services 4
2.1. REST Endpoints 4
2.1.1. Additional Information 4
2.2. Digital Signatures 8
3. Interpreting the Responses 9
3.1. Validation Errors 9
3.1.1. Request Validation 9
3.1.2. VAT3 Return Validation 9
3.2 VAT3 File Web Service 10
4. Digital Signatures 11
4.1. HTTP Signatures 11
4.1.1. HTTP Signature Sample 11
4.1.2. HTTP Signature Components 11
4.1.3. Signature String Construction 12
4.1.4. Signature Creation 14
5. Example Messages 14
6. Public Interface Testing 14
Appendix A – Extracting from a .p12 File: 15

Audience

This document is for any software provider who has chosen to build or update their products to allow for VAT3 Form submissions via the new VAT3 Webservice.

Document context

This document provides a technical overview of how to integrate with Revenue's REST web services including how to sign requests validly. This document is designed to be read in conjunction with the REST/XML example files as well as the rest of the Revenue Commissioners' VAT3 Webservice documentation suite including the relevant technical documents.

Related Documents

This document should be read in conjunction with the following documents:

Document Name	Description
VAT3 Return Rules	Outlines the business rules and validation logic that govern a valid VAT3 return submission.
VAT3 Schema Notes	Describes the schema structure for the VAT3 request, covering the expected elements, data types, and an example XML file.
VAT3 Webservice Error Structure	This document provides details of REST error message structure for the VAT3 Webservice.
Sample HTTP messages	Example files of http response messages.

1. Introduction

This document details the REST VAT3 web services specification for the following web services:

- VAT3File endpoint – a webservice for VAT3 return submissions.

2. Calling the Services

The OpenAPI, REST Security version specifications we are following include:

Open API Specification Version 3.0: https://github.com/OAI/OpenAPI-Specification/blob/master/versions/3.0.0.md

HTTP Signatures Version 08: https://tools.ietf.org/html/draft-cavage-http-signatures-08

2.1. REST Endpoints

The VAT3File web service endpoint is detailed below.

Description	HTTP Method	Endpoint URL	Additional Information	Links
* Submit a VAT3 XML file via the VAT3File endpoint to submit a VAT3 return.	POST	https://www.ros.ie/VAT3/File	Query Parameters: ○ softwareUsed. ○ softwareVersion. ○ AgentTain (optional*).	

2.1.1. Additional Information

Schema Overview:

This section contains all the validation rules, which must be passed to enable a successful form upload to ROS for VAT3, Supplementary VAT3 and Amended VAT3.

These include for each attribute or element:

- Correct data formats.
- Maximum and minimum values where applicable.

This document should be used as an aid to the VAT3 v1.5 Form schema and should be read in conjunction with the VAT3 Webservice Error Structure and Sample HTTP Messages documents. It details the data types and defaults for each of the elements and attributes within the schema. The XML message must be encoded using UTF-8. The first line within the XML message must indicate this.

VAT3 Payload Parameters:

Name	Type	Minimum Value/ Length	Maximum Value/Length	Required (Y/N)	Description / Validation
name	Text Character	1	30	Y	Trading Name. Up to and including 30 characters in length.
regnum	Text Character	8	9	Y	The VAT Registration Number of the customer (length 8 or 9). Required format is 7 numerics (including leading zeros) followed by one or two letters.
filefreq	Numeric	0	3	Y	Filing Frequency: 0 for Bi-Monthly / Repayment Only 1 for Other 2 Four Monthly 3 Bi-Annual
startdate	Date	N/A	N/A	Y	Begin Date of the VAT3 Period (DD/MM/YYYY).
enddate	Date	N/A	N/A	Y	End Date of the VAT3 Period (DD/MM/YYYY).
type	Numeric	0	2	Y	Type of VAT3 Return: 0 for an Original VAT3 Return 1 for a Supplementary VAT3 Return 2 for Amended VAT3 Return
sales	Numeric	0	999999999	Y	VAT on Sales in Euro only. (No cent) Maximum of €999,999,999

purchs	Numeric	0	999999999	Y	VAT on Purchases in Euro only. (No cent) Maximum of €999,999,999
goodsto	Numeric	0	999999999	Y	Total goods to other EU countries in Euro only. (No cent) Maximum of €999,999,999
goodsfrom	Numeric	0	999999999	Y	Total goods from other EU countries in Euro only. (No cent) Maximum of €999,999,999
servicesto	Numeric	0	999999999	Y	Total services to other EU countries in Euro only. (No cent) Maximum of €9,999,999,999
servicesfrom	Numeric	0	999999999	Y	Total services from other EU
formversion	Numeric	1	1	Y	Must be equal to '1' for this version of the file format.
language	Text Character	1	1	Y	Language through which the return is being filed: - E for English, or - G for Irish
currency	Text Character	1	1	Y	The currency through which the return is being filed: - E for Euro, or This should always be 'E' for Euro as ROS will not accept VAT3 Returns in Punt.
unusualExpenditure	Text Character	1	1	N	Flag indicating presence of unusual expenditure: - Y for yes - N or no entry for no
unusualExpenditureAmt	Numeric	0	999999999	N	Amount of unusual expenditure Not permitted if unusualExpenditure field is not set to Y

unusualExpenditureDtl	Text	0	50	N	Description of unusual expenditure Not permitted if unusualExpenditure field is not set to Y
postponedAccounting	Numeric	0	999999999	Y	Value of goods imported under Postponed Accounting (net plus carriage, insurance, and freight (CIF)) If a value is entered for postponedAccounting, values of sales and purchs should include values greater than 0.

Sample XML File:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>

<VAT3 currency="E" name="Trade Name" regnum="1234567T" startdate="01/01/2021"
enddate="28/02/2021" sales="999999999" purchs="999999999" goodsto="999999999"
goodsfrom="999999999" servicesto="9999999999" servicesfrom="999999999" type="0"
filefreq="0" formversion="1" language="E" postponedAccounting="9999999999"
unusualExpenditure="Y" unusualExpenditureAmt="999999999" unusualExpenditureDtl="Unusual
Expenditure"/>
```

VAT3 Response Parameters:

Name	Type	Minimum Value/ Length	Maximum Value/Length	Required (Y/N)	Description / Validation
Id	string	1	1	Y	A unique identifier for the response message.
Status	string	1	1	Y	The processing status. Must be one of: 'Processed', 'Invalid', or 'Error'.
ErrorList	ErrorCode	0	unbounded	N	A container for one or more error messages. Present only if the Status is 'Invalid' or 'Error'.
Code	int	1	1	Y	A numeric code identifying the specific error.
Message	string	1	1	Y	A human-readable message describing the error.

2.2. Digital Signatures

The VAT3 File web service will require a digital signature. This will be the digital signature of the declarant.

3. Interpreting the Responses

The VAT3 web service will return a response message to the client as outlined below.

3.1. Validation Errors

3.1.1. Request Validation

When a request is made to the VAT3 web service three checks are carried out before any processing can occur. These include:

1. Authentication
2. Authorisation
3. XML API validation

Step 1 of the validation process tries to verify the authenticity of the message by checking it has been signed with a valid digital signature. Step 2 focuses on authorising the credentials and finally, step 3 checks the message is valid using XML API validation.

If there are any errors encountered during the above processes, the response will include a HTTP status code of either 401 (Unauthorised) if authentication fails, 403 if authorisation fails and 400 (Bad Request) if XML API validation fails. The message body will provide more information on the details of the problem. Where an error code is returned to the client, no processing will occur for that message.

If the message passes XML API validation, authentication, and authorisation but the resource cannot be found the response will include a HTTP status code of 404. If the resources query parameters are not as required or are missing, the response will include a HTTP status code of 400. Where an error code is returned to the client, no processing will occur for that message.

3.1.2. VAT3 Return Validation

The user submits a VAT3 file to the web service endpoint: /file

After receiving the request, the system will validate the data internally:

- Schema validation
- VAT3 business rules (same process as ROS offline upload).
- Verification of details

Based on the validation:

- If the payload passes all checks, it will be submitted and inserted into the database, including any refund or payment processing.
- If there are issues, the system will respond with an error message.
- An acknowledgement with a success or error message will be returned to the user.

3.2 VAT3 File Web Service

The VAT3 File response returns an Acknowledgement with either a success or an error message.

If validation failed or there is an issue with the submitted schema, a list of the errors is returned in this response.

4. Digital Signatures

Any ROS web service request that either returns confidential information or accepts submission of information must be digitally signed. This must be done using a digital certificate that has been previously retrieved from ROS.

The digital signature must be applied to the message in accordance with the HTTP Signatures specification as specified in Section 4.1.

The digital signature ensures the integrity of the document. By signing the document, we can ensure that no malicious intruder has altered the document in any way. It can also be used for nonrepudiation purposes.

If a valid digital signature is not attached, a HTTP status code of 401 (Unauthorised) will be returned. The message body will provide more information on the details of the problem.

4.1. HTTP Signatures

The HTTP signatures protocol is intended to provide a simple and standard way for clients to sign HTTP requests. A summary of the structure of a HTTP Signature is outlined below. This is a simplified explanation of the HTTP Signatures specification. The full specification can be found at Signing HTTP Messages and should be read in full. The specification defines two approaches to building a HTTP signature, "The 'Signature' HTTP Authentication Scheme" and "The 'Signature' HTTP Header", Revenue uses the latter.

At a high level, a HTTP Signature is a HTTP header that is added to a HTTP request. It is comprised of a set of components that were used to generate a digital signature and the digital signature itself.

4.1.1. HTTP Signature Sample

Below is a sample HTTP Signature header.

```
Signature: keyId="MIICfzCCAeigAwIBAgIJ... // truncated",
algorithm="rsa-sha512", headers="(request-target) host date
digest", signature="GdUqDgy94Z8mSYUjr/rL6qrLX/jmudS... //
truncated"
```

4.1.2. HTTP Signature Components

The Signature HTTP header contains four components, keyId, algorithm, headers, and signature. Below is a description of each.

keyId: The keyId field must contain a Base64 encoded version of the X509 certificate that accompanies the private key used to sign the message. This field is required.

algorithm: The `algorithm` parameter is used to specify the digital signature algorithm to use when generating the signature. Revenue expects this to be 'rsa-sha512'. This field is required.

headers: The `headers` parameter specifies the list of headers used when generating the signature for the message. The parameter must be a lowercased, quoted list of HTTP header fields, separated by a single space character. The list order is important and MUST be specified in the order the HTTP header field-value pairs are concatenated together during signing. For POST requests, the digest field must be included.

signature: The signature component is a base 64 encoded digital signature. The implementer uses the `algorithm` and `headers` field to form a canonicalized `signing string`. This `signing string` is then signed with the private key that accompanies the X509 certificate associated with the `keyId` field and the algorithm corresponding to the `algorithm` field. The `signature` field is then base 64 encoded.

4.1.3. Signature String Construction

In order to generate the string to be signed, the implementer MUST use the values of each HTTP header defined in the `headers` signature field, to build the signature string. Values must be in the order they appear in the `headers` signature field. If the associated HTTP header does not exist, it should be added to the HTTP request BEFORE attempting to construct this string.

Date:

The GMT time zone is enforced. Requests expire after 90 minutes has elapsed based on the timestamp received on the date/x-date header. Requests made up to 90 minutes before the revenue server time are also accepted to counteract the potential for server time discrepancies and daylight savings. For example:

A message with the timestamp in ISO_8601 ("yyyy-MM-dd'T'HH:mm:ss.SSSX") format is received as follows:

2018-01-01T12:00:00.000Z

This message is valid for 90 minutes either side of it.

i.e., from 2018-01-01T10:30:00.000Z to 2018-01-01T13:30:00.000Z

Values for days and months should be two digits. As such, in the case where either the day or the month is a single digit, such as 2018-1-1, the single digit value should be prepended with a zero to make it 2018-01-01.

The accepted formats for date are:

- ISO 8601
- RFC 822, updated by RFC 1123
- RFC 850, obsoleted by RFC 1036
- ANSI C's time format

Allowable values in the headers field are outlined in the table below.

Value	Mandatory
* (request-target)	Yes
host	Yes
date	Yes
** x-date	Yes, if date header cannot be added.
*** digest	Yes, if HTTP method is of type POST
**** content-type	No
content-length	No
x-http-method-override	If HTTP method is of type POST, HTTP header *'X-HTTPMethod-Override'* exists and *'Content-Type=application/xwww-form-urlencoded'.* See Section 2.1.1 for more detail.

* The `(request-target)` header field is a special header field in that its value is comprised of 2 HTTP headers. It is generated by concatenating the lowercase HTTP method, an ASCII space, and the request path headers. See below for sample:

(request-target): post /vat3/file/?softwareUsed=XYZ&softwareVersion=1.0

** The 'x-date' headers field value should ONLY be used in conjunction with the X-Date HTTP header if a Date HTTP header cannot be added to the HTTP request programmatically. The Date header has a limitation when using javascript in a browser to build and send a HTTP signature. The limitation is that you cannot add a 'Date' HTTP header when executing javascript in a browser. The native xmlhttprequest object prohibits addition of a 'Date' HTTP header. Building the signature string that will be signed with an 'x-date' header instead of a 'date' header removes this restriction.

*** The 'Digest' HTTP header is created using the POST body/payload. The payload should be converted to a byte array, hashed using the SHA-512 algorithm and finally base64 encoded before adding it as a HTTP header.

**** Although Content-Type does not have to be part of the Signature string, it is required as a HTTP Header if HTTP Method Type is POST. If the request is a standard POST (i.e. not a header *'X-HTTP-*

`Method-Override`'POST, then the Content-Type should be set to 'application/xml' or 'application/xml;charset=UTF-8'

All other header field values are created by concatenating the lowercase header field name followed by an ASCII colon `:`, an ASCII space ` `, and the header field value. Leading and trailing whitespace in the header field value MUST be omitted. If the header field is not the last value defined in the `headers` signature field, then append an ASCII newline `\n`. See sample below.

```
(request-target): post/vat3/file/?softwareUsed=XYZ&softwareVersion=1.0 host: softwaretestnextversion.ros.ie
date: yyyy-MM-ddTHH:mm:ss.SSSX
content-type: application/xml; charset=utf-8
```

4.1.4. Signature Creation

The signature component is a base 64 encoded digital signature. The implementer uses the `algorithm` and constructed Signature String. The Signature String is signed with the private key that accompanies the X509 certificate associated with the `keyId` field and the algorithm corresponding to the `algorithm` field. The `signature` field is then base 64 encoded.

5. Example Messages

There is an adjoining file 'REST_Web_Service_Integration_Guide_Sample_Http_Messages.txt' that contains HTTP Request/Response samples describing the structure of the request and outlining the possible responses a client can expect. Monetary figures in all examples are for illustrative purposes only.

6. Public Interface Testing

For Public Interface Testing, any parties will need to request certificates to the PIT3 environment through email (ROSDevSupport@revenue.ie).

This service will be available through the following URL - https://softwaretest.ros.ie/vat3-webservice-v1/file

Appendix A – Extracting from a .p12 File:

Each customer of ROS will have a digital certificate and private key stored in an industry standard PKCS#12 file.

In order to create a digital signature, the private key of the customer must be accessed. A password is required to retrieve the private key from the P12 file. This password can be obtained by prompting the user for their password.

The password on the P12 is not the same as the password entered by the customer. It is in fact the MD5 hash of that password, followed by the Base64-encoding of the resultant bytes.

To calculate the hashed password, follow these steps:

1. First get the bytes of the original password, assuming a "Latin-1" encoding. For the password "Password123", these bytes are: 80 97 115 115 119 111 114 100 49 50 51(i.e., the value of "P" is 80, "a" is 97, etc.).
2. Then get the MD5 hash of these bytes. MD5 is a standard, public algorithm. Once again, for the password "Password123" these bytes work out as: 66 -9 73 -83 -25 -7 -31 -107 -65 71 95 55 -92 76 -81 -53.
3. Finally, create the new password by Base64-encoding the bytes from the previous step. For example, the password, "Password123" this is “QvdJref54ZW/R183pEyvyw==”.

This new password can then be used to open a standard ROS P12 file.